

The code in the property's getter, setter, and deleter methods acts on the backing variable directly. You don't want the getter, setter, or deleter methods to act on the property, because this could cause errors. In one possible example, the getter method would access the property, causing the getter method to call itself, which makes it access the property again, causing it to call itself again, and so on until the program crashes. Open a new file editor window and enter the following code, saving it as *badPropertyExample.py*:

```
class ClassWithBadProperty:
    def __init__(self):
        self.someAttribute = 'some initial value'

    @property
    def someAttribute(self): # This is the "getter" method.
        # We forgot the _ underscore in `self._someAttribute here`, causing
        # us to use the property and call the getter method again:
        return self.someAttribute # This calls the getter again!

    @someAttribute.setter
    def someAttribute(self, value): # This is the "setter" method.
        self._someAttribute = value

obj = ClassWithBadProperty()
print(obj.someAttribute) # Error because the getter calls the getter.
```

When you run this code, the getter continually calls itself until Python raises a `RecursionError` exception:

```
Traceback (most recent call last):
  File "badPropertyExample.py", line 16, in <module>
    print(obj.someAttribute) # Error because the getter calls the getter.
  File "badPropertyExample.py", line 9, in someAttribute
    return self.someAttribute # This calls the getter again!
  File "badPropertyExample.py", line 9, in someAttribute
    return self.someAttribute # This calls the getter again!
  File "badPropertyExample.py", line 9, in someAttribute
    return self.someAttribute # This calls the getter again!
  [Previous line repeated 996 more times]
RecursionError: maximum recursion depth exceeded
```
